

P2360R0: Extend *init-statement* to allow *alias-declaration*

Introduction

C++98 and onwards allowed to declare a local variable in a for loop:

```
for (int i = 0; i < 10; ++i)
    /* something */;
```

The papers

- P0305R1 Selection statements with initializer and
- P0614R1 Range-based for statements with initializer

extended this facility such that additional variables can be declared in if, switch, and range-based for statements.

Grammatically, this is an *init-statement*, which allows expressions and variable declarations as well as typedefs, but (inconsistently) not *alias-declarations*. This paper proposes to also allow *alias-declarations*.

C++20	this paper
<pre>for (typedef int T; T e : v) /* something */;</pre>	<pre>for (using T = int; T e : v) /* something */;</pre>

Rationale

This is an obvious inconsistency. For modern code, it is generally recommended to use *alias-declarations* in lieu of typedefs (e.g. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rt-using>). The example shown in the introduction is the only case where this fails.

The alternative would be to prohibit typedefs as an *init-statement*, but while unusual, they seem useful in some situations. After all, the argument of a reduced scope for variables introduced in an *init-statement* applies to typedefs as well.

Miscellanea

No feature test macro is proposed. If a user cares about backward compatibility, a typedef should be written unconditionally. The argument for a #error check seems weak, since the compiler error message for an unrecognized *alias-declaration* is unconditional and should be clear.

This has not been implemented.

Wording

Change in 8.1 [stmt.pre] paragraph 1:

init-statement:
expression-statement
simple-declaration
alias-declaration